Software Testing

Testing Models, Testing Types, and Code Review

Saqif Kaisar

Department of CSE, UIU

CSE 3421 – Software Engineering

Why This Lesson?

No customer likes to buy **faulty products**.

The same goes for the **software industry**.

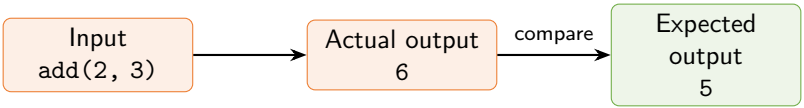
Simple Example – An Online Shop

A customer orders one book but is charged twice. Even if the website looks good, its incorrect behavior damages the customer's trust.

What Is Software Testing?

Definition

Software testing is the process of analyzing a software item to detect the differences between **existing and expected conditions** (i.e., bugs).



Simple Example

The actual result is different from the expected result: the test reveals a bug.

When Should We Test Software?

Throughout the **whole development process**.

Before and During Coding

- Review requirements and design.
- Check individual units as they are written.
- Test interactions during integration.

Before and After Release

- Test the complete system.
- Check customer acceptance criteria.
- Re-test after fixes and changes.

Simple Example

For a shopping cart: review discount rules, test the calculation, test checkout, and re-test checkout after adding coupons.

Testing Models

Black Box Testing

Test externally visible behavior without examining the internal mechanism.

White Box Testing

Test with knowledge of the internal mechanisms and code.

One Feature, Two Views

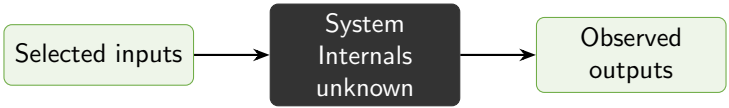
Black box: Does a valid password allow login?

White box: Have both the successful-login and failed-login branches executed?

Black Box Testing – Main Idea

Definition

Ignores the internal mechanism of a system or component and focuses solely on the outputs generated in response to selected inputs and execution conditions.



Simple Example

Enter a wrong password and check that login is rejected. You do not need to read the authentication code.

Black Box Testing – What Do We Test?

- The **interface is visible**; the internals are unknown.
- Test functionality against the specifications from the outside.
- For software, examine the interface:
  - What is input to the system?
  - What can you do externally to change the system?
  - What is output from the system?
- Observe external behavior to test the system's functionality.

Simple Example – Shopping Cart

Input: add two books priced at 100 each. **Action:** remove one book.
Expected output: one book remains and the total is 100.

Black Box Testing – Advantages

- **Robust with respect to implementation changes.**
  - Test data need not change when code changes, if behavior stays the same.
- **Allows independent testers.**
  - Testers need not be familiar with the code.
- Can be **less time-consuming**: no internal-code analysis is needed.

Simple Example

Developers replace the search algorithm. The test "search for an existing book and check that it appears" can remain unchanged.

Black Box Testing – Disadvantages

- May miss implementation bugs not covered by the specification or selected tests.
- Does not directly guide internal performance optimization or the choice of alternate algorithms.

Simple Example

A checkout test confirms the correct total, but does not reveal that the code calculates that total ten times unnecessarily.

Keep in Mind

External tests can measure response time, but inspecting the code helps explain *why* an implementation is inefficient.

White Box Testing – Main Idea

Definition

Takes into account the **internal mechanisms** of a system or component.

- Use knowledge of the internal workings to test what happens inside.
- Examine procedural details closely.
- Test logical paths: **conditionals, loops, and branches**.
- Examine program state against expected behavior.

Simple Example

A discount applies when `total >= 1000`. Test totals of 999 and 1000 to execute both the "no discount" and "discount" branches.

White Box Testing – Advantages

- Allows thorough testing of the software's internal logic.
- Helps find hidden bugs and errors.
- Yields useful test cases.
- Enables code optimization and experiments with alternate algorithms.

Simple Example

Inspecting a cart-total loop suggests testing an empty cart, a cart with one item, and a cart with several items. This can reveal a skipped last item.

White Box Testing – Disadvantages

- **Exhaustive path testing is often impractical.**
  - The number of paths grows rapidly; loops can make it unbounded.
- Requires access to the code.
- Requires a skilled tester with detailed knowledge of the internals.
- More time-consuming because internal logic must be analyzed.

Simple Example

Just 20 independent yes/no decisions can create $2^{20} = 1{,}048{,}576$ possible combinations of outcomes.

Functional Testing

- Tests the system against its **Functional Requirements Specification (FRS)**.
- Tests functions or features by feeding them input and examining output.
- Focuses on the **results of processing**, rather than how processing occurs.
- Commonly uses black box testing: the tester need not know the internal logic.

Simple Example

Requirement: a valid coupon gives a 10% discount.
Test: apply the coupon to a total of 500.
Expected result: the new total is 450.

Non-Functional / Technical Testing

- Checks non-functional aspects such as **performance, usability, reliability, and security**.
- Tests readiness against non-functional parameters, rather than only checking whether features produce correct results.
- Can use white box or black box techniques, depending on the aspect tested.

Simple Example – The Same Checkout Feature

Functional: does checkout calculate the correct total?

Non-functional: does checkout respond within two seconds with 500 simultaneous users?

Types of Testing

Levels of Testing

- Unit / component testing
- Integration testing
- System testing
- Acceptance testing

Other Testing Purposes

- Load and stress testing
- Security testing
- Regression and smoke testing
- Beta testing

Simple Example

Test a price calculation first, then its connection to checkout, then the whole shop, and finally whether the customer accepts it.

Unit Testing

- Tests individual software or hardware units, or groups of related units.
- Done by programmers; generally uses white box knowledge.
- Verifies that code does what it is intended to do at a very low structural level.
- Automation is desirable for repeatability.
- **Component testing:** almost identical, but at a slightly larger scale.

Simple Example

Unit: check that `calculateTotal(100, 2)` returns 200.
Component: test the cart module's add, remove, and total operations together.

Integration Testing

- Combines software components, hardware components, or both to evaluate their interactions.
- Done by programmers as they integrate code into the codebase.
- Verifies that units work together in the larger codebase.
- Individually working components may still fail when combined.
- Generally uses white box knowledge; may also use black box tests.
- Automation is desirable for repeatability.

Simple Example

The cart sends 500 to the payment module, but one module uses whole currency units and the other expects cents. An integration test exposes the mismatch.

System Testing

Definition

Tests a **complete, integrated system** to evaluate compliance with its specified requirements.

- Evaluates the entire system against the functional and/or non-functional requirements.
- Together, these form the **Software Requirements Specification (SRS)**.
- Covers non-functional requirements as well as functional behavior.

Simple Example

Complete a purchase: log in, search, add to cart, pay, and receive a confirmation. Also check the specified response-time requirement.

Load and Stress Testing

Load Testing

- Performance testing under real-life load conditions.
- Uses concurrent users and transactions over a set duration.
- Measures response times and helps identify operating capacity.

Stress Testing

- Evaluates the system beyond its specified limits.
- Increases pressure until performance degrades or failure occurs.
- Helps identify the system's breaking point.

Simple Example

Load: test checkout with the expected 500 concurrent users.
Stress: increase to 1,000, 2,000, and more to find where it fails.

Load vs. Stress – The Key Difference

	Load Testing	**Stress Testing**
Goal	Handle expected volume with acceptable degradation	Find behavior beyond normal limits
Conditions	Normal, expected load	Extreme load, possibly up to failure
Question	"Does it meet the target?"	"When and how does it break?"

Intuitive Analogy

Load: check a lift carrying its rated number of passengers.
Stress: in a controlled test, exceed that rating to examine its limits.

Security Testing

Purpose

Uncover vulnerabilities and determine whether data and resources are protected from possible intruders.

- **Vulnerability scanning:** automated scanning against known vulnerability signatures.
- **Ethical hacking:** authorized attempts to expose security flaws rather than exploit them for personal gain.
- **Penetration testing:** a controlled, simulated attack to identify and assess exploitable vulnerabilities.

Simple Example

In an authorized test environment, sign in as Customer A and try to view Customer B's order. The system should deny access.

Regression Testing

Definition

Selective retesting to verify that modifications have not caused unintended effects and that the system still meets its specified requirements.

- Re-run a core group from the original tests after significant changes.
- Choose representative tests covering existing functionality.
- Add tests for areas most likely to be affected by the change.
- **Smoke test:** a small set of checks, often drawn from the regression suite, establishing basic stability and major functionality under normal conditions.

Simple Example

After adding coupons, re-test totals, payment, and refunds. A quick smoke test checks that login, adding an item, and basic checkout still work.

Beta Testing

An organization can offer an advance partial or full version, often free, to potential users. They use it freely and report errors they encounter.

Advantages

- Identifies unexpected errors.
- Low cost.
- Wider population and environments.

Disadvantages

- Lack of systematic testing.
- Error reports may be low quality.

Simple Example

A small group tries a shopping app before release. One user discovers that checkout fails on an older phone, but reports only "payment not working."

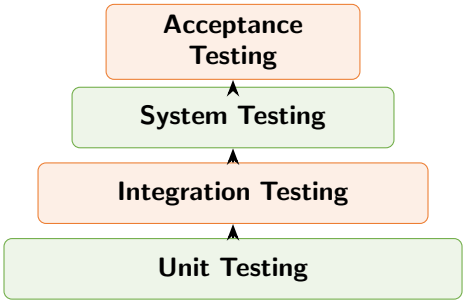
Acceptance Testing

- Formal testing to determine whether the system satisfies its **acceptance criteria**.
- Enables the customer to decide whether to accept the system.
- Generally performed by the customer or a representative in their environment, through the GUI.
- A black box activity focused on externally observable results.

Simple Example

A shop owner checks the agreed criteria: customers can place an order, stock decreases correctly, and a receipt is produced. Passing these checks supports the acceptance decision.

Testing Hierarchy



Increasing Scope

Individual units → interactions → whole system → customer acceptance.

- **Regression:** repeat relevant tests after changes, at any level.
- **Beta:** gather real-user feedback before general release.

Simple Example

Price function → cart and payment → full shopping journey → shop owner's acceptance checks.

Static and Dynamic Testing

Static Testing

- Examines something that is **not running**.
- Reviews the work rather than executing it.
- Examples: code review and software inspection.

Dynamic Testing

- Works with **running software**.
- Supplies inputs and checks expected outputs.
- Examples: unit, integration, system, and acceptance tests.

Simple Example

Static: read the discount code and notice a wrong comparison.
Dynamic: run checkout with a qualifying total and observe no discount.

Code Smell

- Code components that violate sound development principles and reduce code quality.
- Negatively affect **readability and maintainability**.
- Usually **not bugs**: the code may still function correctly.
- Indicate design weaknesses that can slow development and increase the risk of future bugs or failures.
- Examples: long functions, code duplication, and mysterious names.
- Examples of code analysis tools include CodeGrip, Checkstyle, PMD, FindBugs, and SonarQube.

Simple Example

The same discount formula is copied into three functions. It works today, but a later rule change may update only two copies, causing inconsistent totals.

Code Review

Main Idea

Another developer examines a proposed code change before it becomes part of the main project codebase.

Simple Example

Before merging a checkout change, a teammate notices that it does not reject negative item quantities.

Code Review – Definition and Forms

- Analyzes code written by another developer to judge whether its quality is sufficient for integration into the main codebase.
- The formal variant, **software inspection** or **Fagan inspection**, is a long-established quality improvement practice.
- High cost and formal requirements can limit adoption of inspections.
- Lightweight, informal, and tool-based reviews offer less formal alternatives.

Simple Example

A teammate reviews a small pull request, leaves comments, and approves the revised change without organizing a formal inspection meeting.

Code Review Efforts

Illustrative Review-Effort Assumptions

- Average review effort: **6 hours per week**.
- Approximately **10–15%** of developer time.
- Microsoft example: 50,000 developers at \$50 per hour.

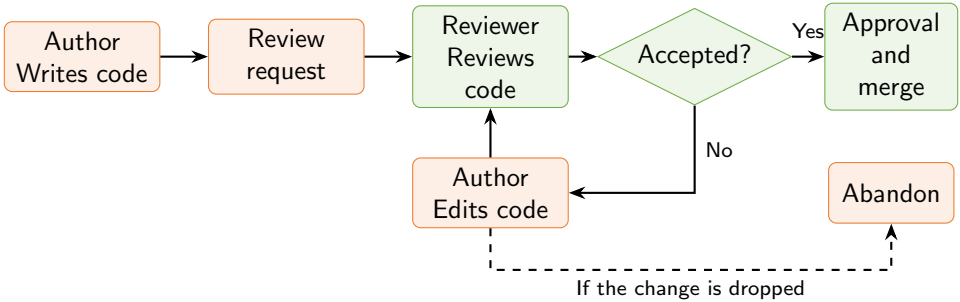
$$50{,}000 \times 6 \text{ hours/week} \times \$50/\text{hour}$$
$$= \$15{,}000{,}000 \text{ per week}$$

Smaller Example

Using the same assumptions, a team of five spends $5 \times 6 = 30$ person-hours, or \$1,500 per week, on reviews.

These illustrative figures are not a current staffing or cost estimate.

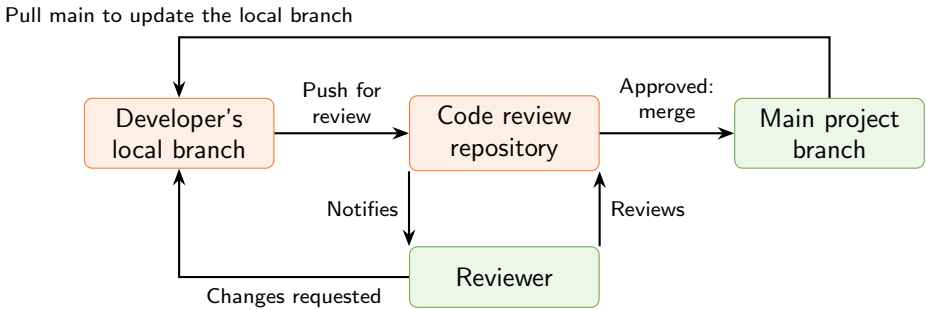
Code Review Workflow



Simple Example

A reviewer requests a negative-quantity check. The author adds it, the reviewer checks the revision, and the approved change is merged.

Simplified Code Review Workflow



Simple Example

Push a checkout fix for review. If changes are requested, edit locally and push again. Merge after approval and update the local branch.

Code Review Tools

Examples of Code Review Tools

- Gerrit
- Review Board
- Phabricator
- Crucible

Simple Example – Tool-Based Review

Open a proposed change, inspect the diff, comment on a missing input check, and review the author's updated version before approval.

Historical tool examples, not a current availability or recommendation list.

Code Review Issues – Readability and Design

1. Coding Style

Code does not follow the team's naming, formatting, or readability conventions.

Example: a new function uses `Calc_total` while the project uses `calculateTotal`. Use consistent naming so the code is easier to read.

2. Redundant Checks

The same condition is checked again even though its result cannot have changed.

Example: inside an `age >= 18` branch, another identical check is made without changing `age`. Remove the unnecessary check.

3. Project Design Violations

Code breaks the project's agreed architecture or separation of responsibilities.

Example: a checkout screen writes directly to the database instead of using the required service layer. Move that operation into the service.

Code Review Issues – Unsafe and Harmful Code

4. Unsafe Methods

An operation is used in a way that can cause unintended execution or damage.

Example: a cleanup function deletes every file in a folder to remove one temporary file. Delete only the intended file instead.

5. Critical Security Defects

A flaw can expose sensitive data or allow unauthorized access.

Example: an order endpoint checks that a user is logged in but not that the order belongs to them. Add an ownership or authorization check.

6. Malicious Code

Code deliberately introduces harmful or unauthorized behavior.

Example: a change secretly sends customer records to an unrelated server. Reject the change and investigate; this is not an ordinary coding mistake.

Code Review Issues – Validation and Correctness

7. Inadequate Input Validation

Input is accepted without checking required formats, ranges, or constraints.

Example: checkout accepts a quantity of -2, producing a negative subtotal. Require a positive integer before calculating the total.

8. Misunderstood Requirements

The code implements a different rule from the one the customer specified.

Example: free delivery applies to totals of 1,000 or more, but the code uses `total > 1000`. Use >= and test the boundary value.

9. Lack of Exception Handling

Failures are not handled appropriately, leaving a crash or an incorrect state.

Example: a payment timeout crashes checkout. Handle the timeout, show a clear message, and confirm payment status before retrying.

Code Review – Practical Impacts

- Peer review can strain interpersonal team relationships.
- Having work critiqued by peers and defect density measured by management can be difficult.
- Managers should create a culture of **collaboration and learning**.
- Each discovered bug is an opportunity to improve code quality, rather than simply a negative judgment.

Simple Example – Focus on the Code

Instead of "You always forget validation," say: "Could we reject negative quantities here and add a test for that case?"

Other Benefits of Code Review

- **Project awareness:** learn what is changing elsewhere in the project.
- **Peer impression formation:** learn colleagues' strengths and approaches.
- **Consistent coding style:** reinforce shared conventions.
- **Knowledge dissemination:** spread understanding across the team.

Simple Example

While reviewing checkout code, a teammate learns how the payment module works and suggests the team's standard naming style. Now more than one developer understands that part of the system.

Thank You

Questions?